Course Registration System

CSC220 / ITE220 — Web Development II

Stamford International University · Trimester 1/2026

Final group project report

Repository: https://github.com/Captansleepy/ITE220-Final-Project-Course-Registration-System

1. Project purpose and scope

The Course Registration System is a full-stack MERN application for managing a university department's users, course offerings, registrations, and academic records. It provides separate dashboards for administrators, academic advisors, and students. The purpose is to support academic decisions with stored data and clear eligibility explanations rather than allow students to register themselves.

Administrators manage accounts. Advisors create course sections, review their assigned students' histories, register eligible students, remove registrations, and control Add/Drop windows. Students view current courses, past grades, earned credits, and outstanding failures. When an Add/Drop window is open, students download a form and email their signed request to the advisor. Opening a request does not automatically change a registration.

2. Architecture and design decisions

The client uses React with Vite and React Router. Components maintain interface state and load data from the API. A shared `client/src/api.js` function centralizes requests, attaches the authentication token, and handles API error messages. Protected routes send users to the dashboard matching their role. Loading and error states distinguish pending requests from successful empty results.

The server uses Node.js and Express, organized into models, routes, controllers, middleware, and utilities. Controllers perform account management, offering management, registration, and dashboard operations. MongoDB Atlas stores persistent data, and Mongoose defines the document schemas, references, and validation. This division separates presentation from academic rules and database access.

Passwords are stored as bcrypt hashes. Successful login returns a JWT identifying the user. Server middleware validates the token, reloads the account, checks whether it is active, and enforces the required role. Frontend route protection improves navigation, while server checks enforce actual access restrictions. Students can read only their own dashboard; advisors can access only their assigned students.

Configuration is kept in local `.env` files. The repository includes `.env.example` files and ignores `.env`, installed dependencies, and generated builds. CORS permits the configured React origin to access the API. No credentials, tokens, or database passwords are included in this report.

3. Database design and dataset

The academic model uses six collections. ObjectId references connect documents instead of duplicating entire user or course records. The accompanying `database-diagram.pdf` shows the collections, fields, and relationships on one page.

Collection	Purpose and relationships
users	Stores account details, password hash, role, and active state. Students have an invented student ID and reference an advisor in users.
courses	Stores unique course codes, titles, credits, and descriptions.
terms	Stores term codes and current/finalized state. A revision field supports registration transaction coordination.
offerings	References a course and term; stores section, meetings, room, instructor, capacity, and Add/Drop settings.
registrations	References a student and offering; records registered or dropped status and timestamps.
records	References a student, course, and term; stores a completed-course grade.

An offering can contain one four-hour weekday meeting or two matching two-hour meetings on Monday/Thursday or Tuesday/Friday. The implementation fixes course credits at four. These constraints match the project's current dataset and schedule configuration; they are project assumptions rather than a general university scheduling model.

The seed validates the workbook and `seed-config.json` before importing. The verified initial import contains 25 students, two advisors, one administrator, 43 courses, ten terms, nine current-term offerings, 237 historical records, and 13 current registrations. These figures describe a fresh seed, not the current shared database. The dataset includes F and W grades as well as passing grades. In-progress courses are stored as registrations rather than completed grade records.

Fresh imports generate demo names and `example.test` email addresses. Student IDs are generated in the form STU001 through STU025, and original advisor assignment mappings are preserved through references. The generated identities separate demonstration accounts from personal identities. The curriculum and grade patterns originate from the project workbook; their institutional accuracy is outside the automated checks.

The importer accepts only a new, empty database whose name starts with `course_registration_seed_`. It refuses a database containing collections and never deletes an existing database. `npm run seed:preview` validates the source without connecting to MongoDB; `npm run seed` performs the import. The configured demo password applies to all seeded accounts. Fresh demo emails include `admin@example.test`, `advisor001@example.test`, and `student001@example.test`.

Academic history is preserved when an administrator deactivates an account. Offerings with registration history cannot be deleted, and their identifying or schedule fields are protected against changes. Seats remaining are calculated from capacity minus active registrations rather than maintained as a separate counter that could become inconsistent.

4. How the registration rules engine works

Eligibility is computed on the server using the selected student's records, active registrations, each offering's term and meetings, and occupied seats. The same rules are checked again when a registration is submitted. This prevents a stale screen or a direct API request from bypassing the checks.

The engine first checks that the offering belongs to a current, non-finalized term. It then searches the student's records for that course. Grades A, B+, B, C+, C, D+, and D count as passes and exclude the course. An F produces a Retake required flag if there is no passing record for the same course. A W does not count as a pass and permits the course to be taken again. Retake offerings are sorted ahead of other offerings in the advisor's list.

The engine compares the offering with registrations in the same term. It blocks duplicate registration for the same course, including another section, and checks every meeting pair for timetable overlap. Meetings clash when they share a day and each starts before the other ends. Adjacent classes whose end and start times are equal are allowed. A clash explanation names the existing course and section.

If the offering passes the previous checks, the engine compares occupied seats with capacity. A full section is blocked with a Full explanation. The advisor sees unavailable offerings and their reasons rather than losing them from the interface. Reasons include Already passed, Already registered, Full, and a named timetable clash. A failed course without a current offering cannot be selected; outstanding failures remain visible on the student dashboard.

Before saving an eligible registration, the advisor sees a confirmation identifying the student, course, section, and term. The backend uses a MongoDB transaction and writes to a shared term revision document to serialize overlapping registration changes. It also touches the student and offering so concurrent account or offering changes can conflict with the transaction. A rejected operation does not consume a seat. Dropping a course changes its status to dropped; re-registering reuses that row and preserves history.

5. Add/Drop and student academic information

An advisor opens an offering's Add/Drop window with a future closing date. The student dashboard displays open or closed status and the deadline in Bangkok time. An expired window or finalized term prevents the request option from remaining open.

The request screen provides the fillable PDF, instructions, the assigned advisor's email, and the subject `Add/Drop Request - <Student ID> - <Course Code>`. The email link prepares the address and subject. The student fills and signs the form and attaches it manually. The advisor makes any approved change through the registration interface. No outgoing email or approval is automated.

Students also see completed records grouped by term, full current section details, and outstanding failures. Earned credits count each passed course once, including a course passed after an earlier failure, so repeated attempts do not inflate the total.

6. Division of work and collaboration

All five members completed their assigned responsibilities. After finishing their own tasks, or whenever they had time available, members helped each other with implementation, debugging, and integration. This cooperation supported the completion of the project across the frontend, backend, and database.

Member	Student ID	Main responsibility
Rachaphon Bhetcharut	2306020001	Team lead: repository, reviews, merging, integration, delivery.
Sham Bahadu	230710005	Backend API, authentication, validation, registration rules, integration; final Add/Drop flow and window controls.
Panruethai Thubthimthong	2303080018	Database models, dataset, seeding, and relationships.
Hathairat Deesrisuk	2105110008	Admin and Advisor interfaces.
Trithep Sik	2406100007	Login, Student interface, and shared components.

The team used feature branches and pull requests to integrate its work. Testing was a shared team effort. The team lead coordinated testing, and the backend developer checked the complete application before changes were merged. Members helped verify the combined frontend, API, and database behavior and resolve issues in their assigned areas.

7. Technical problems and solutions

The implementation addresses several technical risks. Simultaneous registration requests could otherwise overfill a section or bypass timetable checks; transactions and the shared term revision serialize them. Simultaneous administrator changes could leave no active administrator; updates and deactivation use a shared database guard and recheck the remaining administrator count inside the transaction.

Deleting users or used offerings would break academic references. Soft deactivation and offering-history restrictions preserve those references. Confusing in-progress courses with completed grades would distort academic summaries; separate registrations and records keep their meanings clear. Predictable demo emails make a fresh installation easier for a marker to test without inspecting the workbook.

During isolated acceptance testing, an overly long generated database name was rejected by Atlas. A compact timestamp and random suffix resolved the naming problem. A network-restricted test environment also required explicit external access for the Atlas checks. The existing shared database was not used for acceptance mutations.

8. Testing and evaluation

Recorded verification passed 38 backend tests, the frontend production build, lint, and seed preview. Ten groups of live Atlas acceptance checks also passed after a fresh isolated seed. They covered reference consistency; real password login and roles; persistent account/offering changes; retakes and academic summaries; full, passed, and clashing registrations; Add/Drop status; dropping and re-registration; finalized-term restrictions; inactive login; simultaneous one-seat requests; and simultaneous administrator deactivation.

The team lead coordinated testing, and the backend developer checked the complete application before merges, with support from the other members. The team completed all manual checks: login and role restrictions; account and offering management; eligibility explanations; registration, dropping, and seat counts; academic history and earned credits; Add/Drop windows and deadline display; and the student request workflow. The PDF was downloaded, filled, saved, and reopened, and the email link's advisor address and subject were checked. Demo email addresses are fictitious, so the email check verified the request setup rather than delivery to a real advisor. The lecturer has GitHub repository access, and presentation preparation is handled outside the repository.

9. Academic integrity and AI assistance

Codex AI assistance was used to inspect requirements and code, create this report and other documentation, generate the database diagram, capture anonymized dashboard screenshots, add registration confirmation, update demo seeding, and implement and run acceptance checks. Verification consisted of the executed backend tests, frontend build and lint, seed preview, and real Atlas acceptance checks described above.

All members completed their assigned work and supported teammates when available. The team lead coordinated application testing, the backend developer checked changes before merging, and all members supported verification and fixes in their assigned areas. Team members reviewed AI-assisted work and remain responsible for explaining their contributions during the presentation.

10. Supporting materials and sources

The implementation is supported by three dashboard screenshots, the one-page database diagram, a fillable Add/Drop form, setup instructions, the seed script, and automated and live acceptance checks. Together, these provide reproducible evidence of the implemented academic workflow.

Sources used for this report:

- CSC220 Final Project Brief and Rubrics, supplied course document: functional requirements, data requirements, deliverables, and assessment criteria.
- Project README and Git history: architecture, recorded roles, setup, and collaboration evidence.
- `server/src/models`, `server/src/controllers`, and `server/src/utils/registrationRules.js`: database design, authorization, and eligibility behavior.
- `server/seed.cjs`, `server/seed-config.json`, and the source workbook: initial dataset and schedule validation.
- `server/tests` and `server/scripts/acceptance.mjs`: test coverage and live verification results.
- `client/src`, `client/public/forms/add-drop-request.pdf`, and `docs/database-diagram.pdf`: interface, request paperwork, and database relationships.

React, Vite, React Router, Express, Mongoose, MongoDB Atlas, bcryptjs, JSON Web Token utilities, and SheetJS are the principal third-party technologies used by the project. Package versions and dependencies are recorded in the repository manifests and lockfiles.